Designing and simulating a computer

Guided project (GP) – PART I

Objective

To simulate a quasi-design of miniaturized version of TOY i.e., TOY-8 using Logisim.

Context

In this guided project (GP) you are required to implement a computer (CPU) TOY-8 which is a miniaturized version of imaginary computer TOY that we discussed and referred to, in class, extensively. For this GP Logisim – a simulator – is selected. To build that ground up you need to work with few basic abstractions like dot represent a power source, line represents a wire and a cross between lines represents a transistor (switch). When a line is live/hot it is little bold otherwise it'll appear normal (thin). You would not rely on built-in components in Logisim unless explicitly allowed.

Components that make up the CPU (TOY-8) are: 1. ALU (adder, AND, XOR), 2. Memory, 3. Register, (R), 4. PC, 5. IR, 6. Control and 7. Clock, involving combinational as well as sequential circuits.

You would work your way up stepwise designing component after component, circuit by circuit, gate by gate and ultimately transistor by transistor, only focusing at a task at hand or risk losing track and get overwhelmed with the information present here. For manageability the GP is divided into two parts wherein PART 1 you work on ALU which are combinational circuits and PART II wherein you implement the memory components involving sequential circuits and wiring all of them.

Along this fascinating journey you would be learning about many concepts and ideas, discovering tools and techniques and buffing not only technical skills but also your social/team making skills.

This effort will be graded equivalent to a weightage of mid-term; every team will appear in viva with their work for assessment. There is a bonus part and implementation of that will earn you 2 brownie points which will be topped up in your total grade if you get full credit here.

Deadlines

Team registration: Thursday Nov. 16, 2023 1800 hours @ https://forms.gle/C131DGytbD5p3JX5A
Softcopy submission on LMS: Friday Nov. 25, 2023 18:00 hours
Viva: Sunday Nov. 26, 2023 (expected) block the whole day as you could be scheduled for any slot of 2 hour. Every member of group is required to take the viva. Viva slot will be schedule a day before the viva and announced on Discord server.

Credits

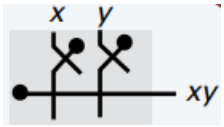
Almost all the diagrams and many examples that are used here are taken from Robert Sedgewick and Kevin Wyne's online course based on their book: Computer Science: An Interdisciplinary Approach. If you are interested and have time you may want to directly look up at the content please visit here: https://introcs.cs.princeton.edu/java/home/ and check Chapter 6 and Chapter 7.

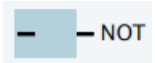
Steps

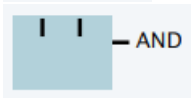
1. Understand the logic behind basic gates: AND, OR, NOT and their derivatives, i.e., NAND, NOR, XOR, XNOR. Study from slides and use Digital design and computer architecture by Sarah and David Harris, hereafter referred as **text**, as supplementary material [ebook uploaded to discord server]
    a. Understanding why NAND and NOR are called universal gates.

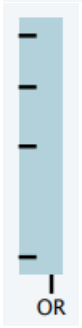
   b. Understanding the XOR can be implemented with two approaches
2. Do a tutorial on Logisim simulator for realizing digital logic circuits' design
   a. Cover a quick beginners tutorial (if you need one) from here: https://youtu.be/cMz7wyY_PxE?si=NGRmEz7m3eUD0dYw
   b. For a guide go here: http://www.cburch.com/logisim/docs/2.7/en/html/guide/index.html
   c. For a complete reference library: http://www.cburch.com/logisim/docs/2.7/en/html/libs/index.html
   You can also equally benefit from the provided tutorial on discord.
3. Understand how switches and transistors realize these logic gates functions; it would be helpful to see how relays are able to control the flow of current first. Use slides and YouTube videos to clear concepts on microchips.
      i. https://youtu.be/RJsAqqObj-Y for getting an overview of building modern chips
      ii. https://youtu.be/J4oO7PT_nzQ?si=pU6Hv8qdTJfgpKWQ for understanding ground up… doping etc.
   b. Make all 7 gates with the help of transistors in Logisim
      i. Lookup for transistors in the Logisim library (URL above) and their use.
   c. Use the following circuit layout diagrams:
      i. 

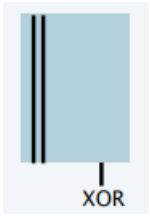
for NOT gate
      ii. 

for OR gate
      iii. 

for AND gate
   d. Do not forget to provide interface diagrams (rectangular diagrams) to the circuit diagrams to neatly fit things in. This is like encapsulation we control complexity through these abstractions.
      i. 

      ii. 

      iii. 

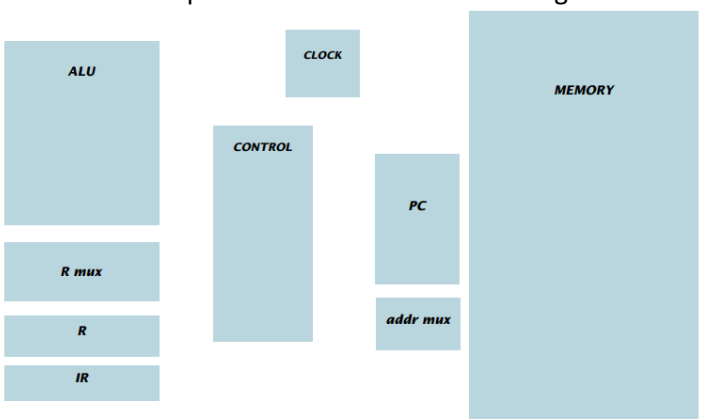
iv. 

4. Understand the De-Morgan's law and its implications. Study this from the **text**.
    a. Every Boolean function can be represented as a sum of product [as we did in class]
    b. Appreciate the key idea that we can use Boolean algebra to systematically analyze circuit behavior [Follow some examples in the text]
    c. To design a circuit that computes given Boolean function
        i. Represent input and output with Boolean variables
        ii. Construct truth table to define a function
        iii. Identify rows where the function is 1
        iv. Use the generalized **AND** for each and **OR** the results
    d. Appreciate that we can design **ANY** circuit with the above approach!
5. List all the components of TOY-8, which are similar to TOY discussed in class albeit it has:
    a. **16** 8-bit words memory
    b. **1** 8-bit register
    c. **1** 4-bit program counter (PC)
    d. **1** instruction type
    e. **8** instructions
6. TOY-8 reference card (Instruction set) is:

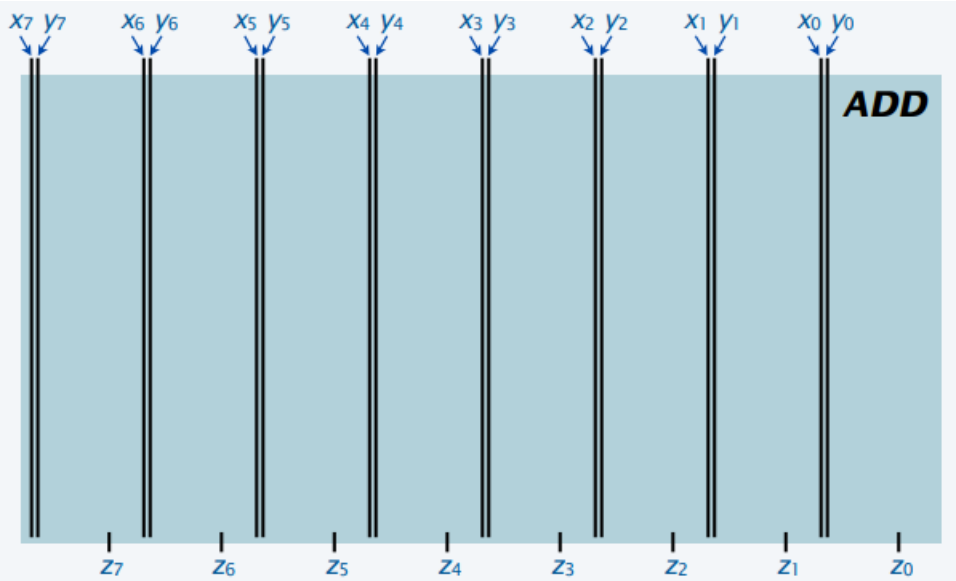
Opcode	Operation	Pseudocode
0	halt	halt
2	add	R = R + M[addr]
4	bitwise and	R = R & M[addr]
6	bitwise xor	R = R ^ M[addr]
8	load addr	R = addr
A	load	R = M[addr]
C	store	M[addr] = R
E	branch zero	If (R = 0) PC = addr

ZERO M[0] is always 0.
STANDARD INPUT Load from M[F]
STANDARD OUTPUT Store to M[F]

7. Circuit modules required to realize the above design are:

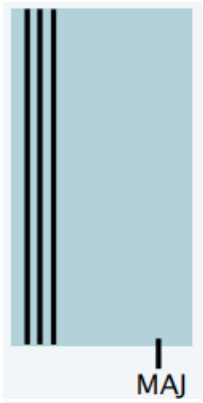


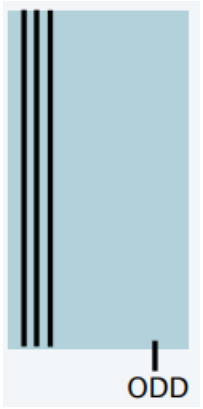
8. Appreciate the difference between busses, data lines, address lines, control lines and modules
    a. Modules are functional components (see 7) consisting of multiple circuits/functions each of which is built with multiple transistors (see 2).
    b. Busses are group of wires that takes input and output for data/addresses, shown here in parallel black lines
    c. Control lines are also essentially wires but used to select different behavior in circuits by selecting different modules to realize a function, shown here in single blue lines.
9. Design/Implement a half adder. Simulate and run
    a. Make a truth table
    b. Observe the behavior for sum and carry bits
    c. Appreciate that some gates can be used to elicit that response/result
10. Design/Implement a full adder (8-bit). Simulate and run
    a. Interface diagram looks like



Where:
z = x + y and,
X_0….X_7, Y_0….Y_7 and Z_0….Z_7 are input and output busses respectively.
Note that we ignore the overflow for now

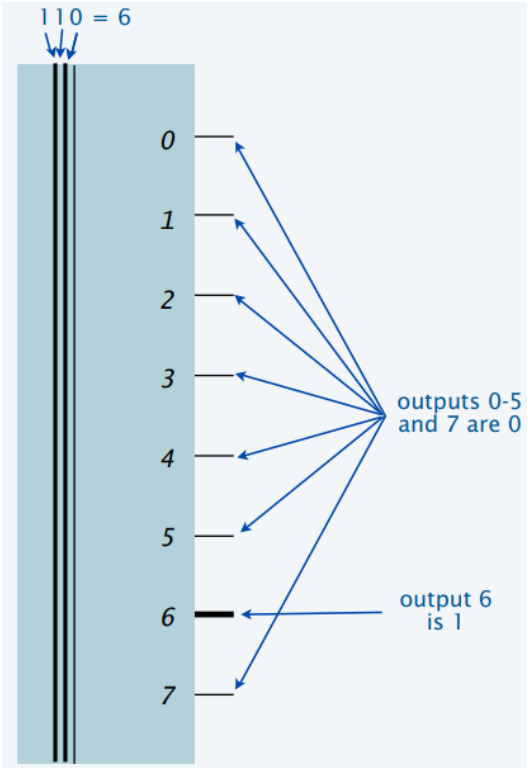
    b. Appreciate that a truth table for this simple 8-bit adder will be more than 65536 rows, so it is an impractical idea! Thus a new design approach is needed.
    c. Do carry bit and sum bit truth tables separately
    d. Appreciate that these results match **MAJ** and **ODD** functions!
    e. Implement **MAJ** and **ODD** functions [Hint: we have done it step-wise in 3c with Boolean functions]
    f. Use the following interfaces:

i. 

ii. 

g. Now chain these **MAJ** to ripple your carries as you would do in math
h. **ADD** will yield the output

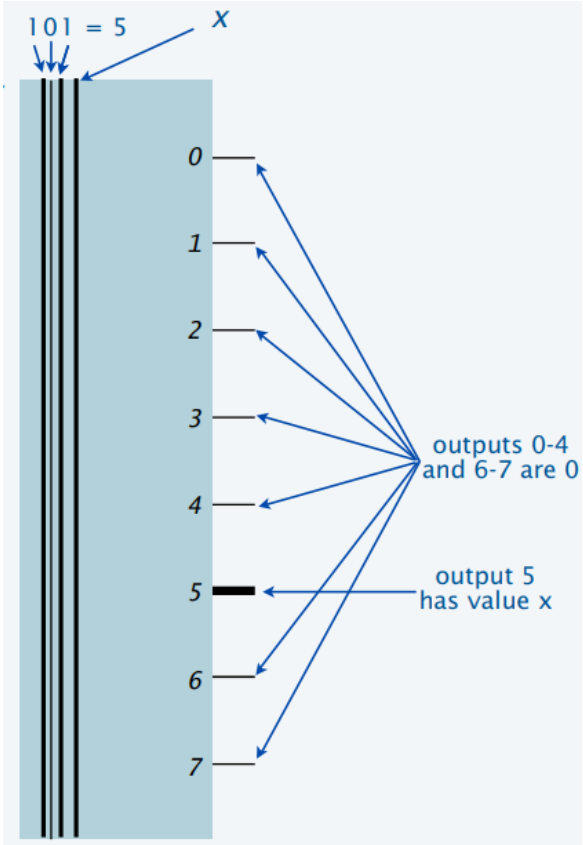
11. Design/Implement a decoder as we would need it
    a. To select a memory word for read/write
    b. Use address bits of instruction from IR
    c. Interface and Example, of 3-to-8 decoder



Where n input lines (address) and 2^n outputs are used, and one and only one output lines will be one all others would be zero.

d. Use 8 (2^n) generalized AND gates for n (3) inputs (Hint: We did an exercise of generalized AND gates for different functions)

12. Design/Implement a demultiplexer [demux], we would need this
    a. To turn on control wires to implement instructions
    b. To use opcode bits of instruction in IR
    c. Interface and example, of 3-to-8 demux

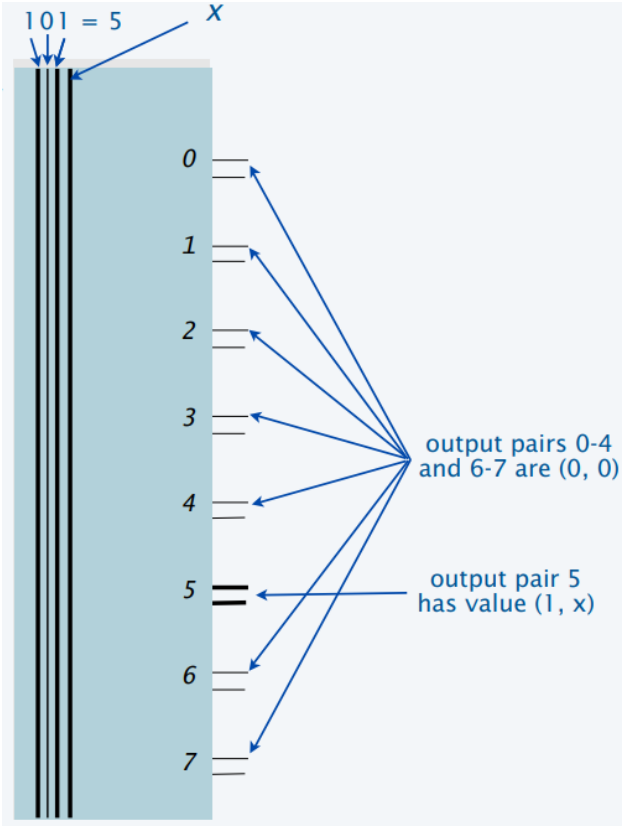


Where n address inputs and 1 data input with value X are used, and 2^n outputs are used. Note that addressed output will get the data value X here, all other outputs will remain 0.

    d. With decoder you have the footprint, use it
    e. Add **AND** X to each gate

13. Design/Implement 3-to-8 decoder/demux we need this
    a. Access and control write of memory word
    b. Use addr bit of instruction in IR

c. Interface and example, of 3-to-8 decoder/demux

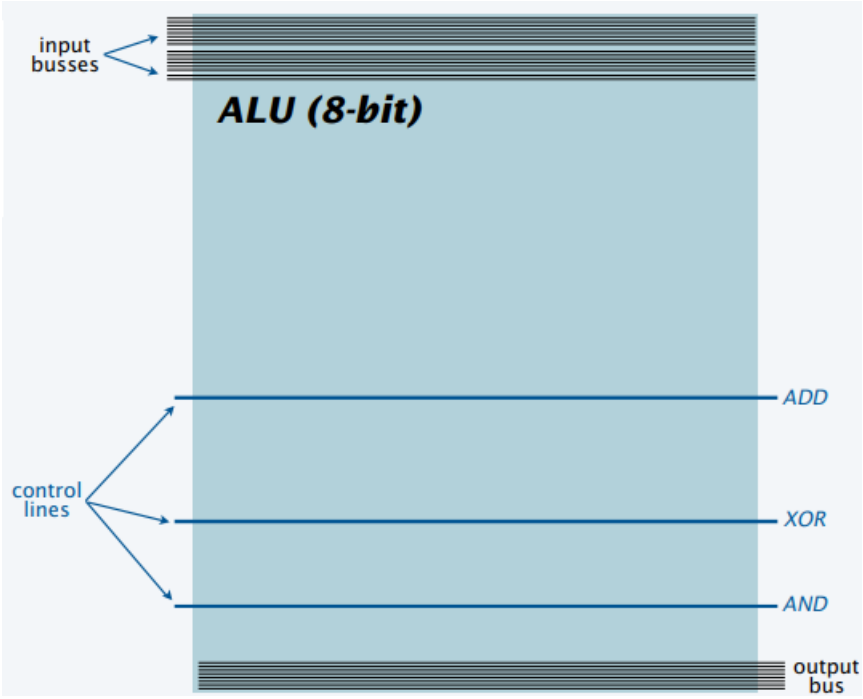


Where n address inputs and 1 data input with value X, 2^n output pairs are used. Addressed output pair has value (1, X). All other outputs are 0.

d. With demux you have the footprint, just add pin of decoder output to demux

14. Design/Implement ALU now

a. Interface diagram will look like



Where 2 input busses and 1 output bus are data-busses and 3 control lines allows us to select different functions such as ADD, XOR and AND for output in ALU, 3

functions however are performed in parallel and a small module (one-hot-mux) allow us to put desired output on output bus.

b. Add earlier built 8-bit ADDER, XOR and AND modules as per configuration
c. Design/implement one-hot-mux, this will allow us to put the result of desired function (ADD, XOR or AND) to output bus
    i. Do bitwise AND to the three functions output such that one input is the data bit and another is the selection line
    ii. Now OR the out of these 3 AND gates against each bit using multi-way OR gate bitwise and connect output of this circuit to output bus
    iii. Attaching these selection lines to a decoder will ensure that only one and one selection is hot/live
    iv. This one-hot-mux is embedded for each bit in our ALU. Note that that it's a virtual selection switch! There aint any connection between input and output.
d. **Left and right shift circuits are omitted those who implement that too would get 2 brownie points**

15. Up until now we explore combinational circuits but to build registers and memory we need sequential circuits (a digital circuit with a feedback loop). Read what sequential circuits are.
    a. Appreciate that memory is the difference between a DFA and a Turing Machine
    b. Memory (bit that can be remembered) is realized with a circuit called flip-flop
    c. **More on it will follow in PART - II**